SOFTLLIGENCE ENTERPRISE CLOUD

Week 1: Backend, Database & System Design

A practical revision sprint with daily implementation, a small inventory API, tests, and a final assessment.

Goal by the end of the week

Build and explain a transaction-safe inventory backend using TypeScript, Node.js, PostgreSQL, automated tests, and a clear architecture note. This is preparation for the larger multi-tenant ERP platform—not a tutorial-copying exercise.

Time commitment: 6 focused study days, roughly 10–12 hours/day. Day 7 is for recovery, review, and catch-up.

Track	What you will demonstrate
Backend engineering	API contracts, validation, error handling, auth concepts, modular structure
Database engineering	Schema design, constraints, indexes, migrations, query plans
Correctness	Transactions, concurrent stock updates, idempotency, integration tests
System design	Tenant isolation, service boundaries, queues, retries, failure modes
Engineering habits	Git commits, reproducible setup, README, ADR, test report

Core rule: you advance when you can implement, test, and explain the concept—not just recognize it in notes.

1. How to use this guide

Use the guide as a working syllabus. Keep a single repository, write your own notes, and make a commit at the end of each day. Read concepts, implement them, test failure cases, then explain the trade-offs in plain English.

Recommended stack

- Runtime: Node.js LTS + TypeScript; choose Express or Fastify and stay consistent for this week.
- Database: PostgreSQL, Docker Compose, and DBeaver or psql.
- Data access: Prisma for migrations and normal queries, plus direct SQL for joins, constraints, and EXPLAIN ANALYZE.
- Validation/API contract: Zod and OpenAPI (document the request and response shapes).
- Tests: Vitest + Supertest or the equivalent for your chosen framework; run integration tests against real PostgreSQL.
- Workflow: Git/GitHub, environment variables, lint/typecheck, and GitHub Actions CI.
- Not yet: Kubernetes, Kafka, premature microservices, and elaborate cloud infrastructure.

Daily time block (adjust if needed)

Block	Focus	Output
2 hours	Concepts and notes	Short notes + questions
2 hours	Hands-on implementation	Working code
2 hours	Database / SQL	Queries, schema, explain plans
2 hours	Tests and debugging	Tests + fixed defects
1–2 hours	System design / DSA	Diagram, trade-offs, or 1–2 problems
30–60 min	Review and documentation	Commit, README update, gap list

Do not force 12 hours of passive reading. If focus drops, take breaks; the goal is durable skill and reliable code.

Repository structure (starter target)

```
softlligence-inventory-api/
  src/
    app.ts
    server.ts
    config/
    modules/
      products/
      warehouses/
      inventory/
      purchasing/
    middleware/
    db/
  prisma/
    schema.prisma
    migrations/
  tests/
    unit/
    integration/
  docs/
    erd.md
    architecture.md
    adr/
  docker-compose.yml
  .env.example
  README.md
```

2. The small project you will build

Project: Inventory & Stock Ledger API

A small backend slice of Softlligence Enterprise Cloud. It should manage products, warehouses, stock movements, and safe stock reservations. Keep it as a modular monolith for Week 1; the goal is to learn correct boundaries before extracting services.

Functional requirements

- Create and list products with SKU, name, and unit; SKU must be unique within a tenant.
- Create warehouses and associate them with a tenant.
- Record stock receipts and issues through a stock-movement ledger; do not silently overwrite stock.
- Reserve stock for an order only when sufficient available quantity exists.
- List stock by product and warehouse, and retrieve a movement history.
- Validate input and return predictable error responses.
- Add tenant-aware authorization to every business query (a simple test identity is acceptable locally; never trust a client-supplied tenant ID in production).
- Write audit-relevant metadata: actor, reason/reference, timestamp, and movement type.

Non-functional requirements

- Integrity: stock must not become negative, including under concurrent requests.
- Isolation: one tenant must never read or mutate another tenant's records.
- Reliability: repeated requests must not accidentally apply a stock change twice when idempotency is required.
- Observability: structured logs, health endpoint, and useful error codes.
- Reproducibility: one documented command starts dependencies; one command runs tests.

Minimum endpoints

Method + path	Purpose	Important checks
POST /products	Create product	Tenant scope, valid SKU, unique constraint
GET /products	List products	Pagination, tenant filter
POST /warehouses	Create warehouse	Tenant scope
POST /inventory/receipts	Receive stock	Positive quantity, transaction, reference
POST /inventory/issues	Issue stock	Sufficient stock, transaction
POST /inventory/reservations	Reserve stock	Atomic availability check
GET /inventory	Read current stock	Tenant + warehouse/product filters
GET /inventory/movements	Audit/history	Pagination, stable ordering
GET /health	Health check	Process status; DB readiness separately if added

Definition of done

- All endpoints have request validation and documented status/error codes.
- Database schema is created by migrations, not manual-only setup.
- Unit and integration tests cover success, invalid input, not-found, forbidden, duplicate SKU, and insufficient stock.
- A concurrency test proves two simultaneous issues cannot oversell the same stock.
- README explains setup, migration, test, API examples, assumptions, and known limitations.

3. Day-by-day plan

Day 1 — Backend fundamentals and API design

Study

- HTTP request lifecycle: method, path, headers, body, status code, content type.
- REST resource design, safe vs idempotent methods, pagination, filtering, sorting, versioning basics.
- TypeScript fundamentals used in backend work: strict mode, types vs runtime validation, unions, async/await, error propagation.
- Middleware and request pipeline; separation of route, controller/handler, service, and data access.
- Authentication vs authorization; session vs JWT concepts; role-based access control (RBAC).
- Input validation, consistent error envelopes, configuration, secrets, and graceful shutdown.

Build

- Create the repository, strict TypeScript config, lint/typecheck/test scripts, and a minimal server.
- Implement GET /health, POST /products, GET /products with in-memory storage only as a temporary step.
- Add Zod schemas, pagination limits, a central error handler, and a consistent JSON error response.
- Write tests for valid input, missing fields, invalid types, duplicate SKU, and unknown routes.

Checkpoint: explain why TypeScript types do not validate untrusted HTTP input at runtime; explain 400 vs 401 vs 403 vs 404 vs 409 vs 500.

Day 2 — Relational data modeling and SQL

Study

- Tables, rows, primary keys, foreign keys, unique/check/not-null constraints, and referential actions.
- Normalization and practical denormalization; one-to-many and many-to-many relationships.
- INNER/LEFT joins, grouping, aggregates, CTEs, transactions, NULL semantics, and ordering.
- Indexes: B-tree basics, composite indexes, selectivity, leftmost-prefix intuition, write cost, and over-indexing.
- Migrations, schema evolution, seed data, and safe changes to production data.

Build

- Start PostgreSQL in Docker Compose and connect with DBeaver or psql.
- Design tenants, users, memberships, products, warehouses, stock balances, stock movements, and idempotency keys.
- Write SQL for stock by warehouse, low-stock products, movement history, and product totals.
- Add constraints and indexes; compare query plans with EXPLAIN (ANALYZE, BUFFERS) on sample data.
- Use migrations to create the schema and seed a small, repeatable dataset.

Checkpoint: show an ERD; explain every foreign key, unique constraint, and index you chose. Do not add indexes without a query or integrity reason.

4. Database design lab

A starter schema (adapt names and types to your ORM). The ledger is the history of changes; a balance table is a performance-oriented current-state projection. If you keep both, update them in the same database transaction and test their consistency.

```
tenants(
  id UUID PRIMARY KEY,
  name TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
)

tenant_memberships(
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  user_id UUID NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('owner','admin','operator','viewer')),
  PRIMARY KEY (tenant_id, user_id)
)

products(
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  sku TEXT NOT NULL,
  name TEXT NOT NULL,
  unit TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, sku),
  UNIQUE (tenant_id, id)
)

warehouses(
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  name TEXT NOT NULL,
  UNIQUE (tenant_id, id)
)

inventory_balances(
  tenant_id UUID NOT NULL,
  warehouse_id UUID NOT NULL,
  product_id UUID NOT NULL,
  quantity NUMERIC(14,3) NOT NULL DEFAULT 0 CHECK (quantity >= 0),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant_id, warehouse_id, product_id),
  FOREIGN KEY (tenant_id, warehouse_id)
    REFERENCES warehouses(tenant_id, id),
  FOREIGN KEY (tenant_id, product_id)
    REFERENCES products(tenant_id, id)
)

stock_movements(
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  warehouse_id UUID NOT NULL,
  product_id UUID NOT NULL,
  movement_type TEXT NOT NULL
    CHECK (movement_type IN ('receipt','issue','reservation','release','adjustment')),
  quantity NUMERIC(14,3) NOT NULL CHECK (quantity > 0),
  reference TEXT,
  actor_user_id UUID,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
)
```

Important design discussion

- Why use NUMERIC rather than floating point for measured quantities? Define a precision/rounding policy for your business units.
- How will you enforce tenant isolation beyond remembering to add WHERE tenant_id = ...? Consider composite foreign keys, authorization in the service layer, and optionally PostgreSQL row-level security after understanding its operational implications.
- Should a reservation reduce physical stock or track reserved quantity separately? For a more realistic design, use on_hand, reserved, and available = on_hand − reserved, with constraints and transactional updates.

- Should stock balances be stored or calculated from all movements? Compare auditability, query cost, consistency, and reconciliation.
- What should happen if a movement insert succeeds but balance update fails? The answer should be one transaction, not a best-effort pair of writes.

SQL exercises

- Find current quantity for every product in each warehouse, including products with zero stock.
- Find the top 10 products by total on-hand quantity for a tenant.
- List stock movements for one SKU in newest-first order with stable pagination.
- Find products below their reorder threshold.
- Use EXPLAIN ANALYZE to compare a query before and after a suitable index.

5. Day 3 — Transactions, concurrency, and idempotency

Study

- ACID: atomicity, consistency, isolation, durability.
- Read committed vs repeatable read vs serializable at a practical level; know that exact behavior is database-specific.
- Lost updates, write skew, race conditions, row locks (SELECT ... FOR UPDATE), and optimistic concurrency.
- Atomic conditional updates; lock ordering; deadlocks and retry policy.
- Idempotency keys for retryable operations; uniqueness constraints as a correctness tool.
- Why a read-check-write sequence can oversell stock when two requests run concurrently.

Build lab: safe stock issue

Do not implement stock issue as “read quantity, check in application, then update” without locking or an atomic condition. Use a transaction and choose one concurrency strategy.

```
BEGIN;

UPDATE inventory_balances
SET quantity = quantity - :qty,
    updated_at = now()
WHERE tenant_id = :tenant_id
  AND warehouse_id = :warehouse_id
  AND product_id = :product_id
  AND :qty > 0
  AND quantity >= :qty
RETURNING quantity;

-- If no row is returned: reject insufficient stock / invalid request.
-- If a row is returned: insert the matching stock movement here.
-- Commit only when both balance and ledger changes succeed.

COMMIT;
```

This is a conceptual SQL pattern. Implement parameterized queries through your chosen data layer, handle missing balance rows, and ensure the movement insert and quantity update are in the same transaction.

Test scenarios

- Quantity 10; issue 4 → final quantity 6 and exactly one issue movement.
- Quantity 10; issue 11 → request rejected and quantity remains 10.
- Quantity 10; two concurrent requests each issue 7 → at most one succeeds; quantity never goes below zero.
- Repeat the same request with the same idempotency key → no duplicate movement.
- Force a failure after the balance update but before movement insert → transaction rolls back both changes.

Day 4 — Maintainable backend architecture and testing

Study

- Modular monolith vs microservices; modules organized around business capabilities.
- Single responsibility, dependency inversion, explicit interfaces, and avoiding abstraction for its own sake.
- DTOs, validation boundaries, service-layer business rules, repositories/data access, dependency injection.
- Unit vs integration vs end-to-end tests; test doubles vs real dependencies; test pyramid trade-offs.
- Configuration validation, secret handling, least privilege, parameterized SQL, CORS/CSRF basics where relevant.

- Structured logs, correlation/request IDs, health/readiness, graceful shutdown, and useful error messages without leaking secrets.

Build

- Move routes/handlers, business logic, persistence, and schemas into separate modules with clear boundaries.
- Write unit tests for pure business rules and integration tests using a real PostgreSQL database.
- Use a test database or isolated schema; tests should clean up after themselves and be repeatable.
- Add CI that installs dependencies, runs lint, typecheck, unit tests, and integration tests.

Checkpoint: you can explain what each test protects, why it would fail if a bug returned, and which bugs require a real database rather than mocks.

6. Day 5 — System design for an enterprise ERP

Use a repeatable design method rather than jumping straight to a diagram.

A 7-step system design method

- 1. Requirements: users, core workflows, permissions, scale, latency, availability, audit and compliance needs.
- 2. API/data model: resources, invariants, access patterns, ownership of records, and tenant boundaries.
- 3. Estimate: active tenants/users, requests per second, read/write ratio, data growth, file/report sizes; state assumptions.
- 4. Components: web/API, identity, domain modules, PostgreSQL, cache if justified, worker/queue for slow work.
- 5. Data flow: request path, transaction boundaries, event/job path, retries, and idempotency.
- 6. Failure analysis: database unavailable, worker crashes, duplicate job, timeout, partial deployment, queue backlog.
- 7. Trade-offs: cost, operational complexity, consistency, performance, security, and team ownership.

Topics you must understand

Topic	What you should be able to explain
Stateless API	Why instances can scale horizontally; where session state lives
Connection pool	Why each request should not open an unlimited new DB connection
Caching	What is cached, invalidation, stale data risk, and when not to cache
Queue + worker	Move slow/retryable work off the request path; jobs are usually at-least-once
Retries/timeouts	Retry transient failures with bounded exponential backoff + jitter; avoid retry storms
Idempotency	Make retries safe; deduplicate by stable business/request key
Rate limiting	Protect shared capacity and isolate abusive clients/tenants
Observability	Logs, metrics, traces; health is not the same as business correctness
Microservices	Independent deployability/data ownership vs network, consistency, and operational costs
Consistency	What must be strongly consistent in a stock transaction vs what can update asynchronously
Security	Tenant authorization, least privilege, secret handling, audit, input validation

Recommended initial architecture

```
Client
  |
API (TypeScript; auth + tenant context)
  |
Modular ERP backend
  |-- Identity / tenant membership boundary
  |-- Catalog + warehouse
  |-- Inventory ledger + reservations
  |-- Purchasing
  |
PostgreSQL (constraints + transactions)
  |
Outbox / job records (only if needed for reliable async events)
  |
Operations worker (later: Redis + BullMQ)
  |-- CSV imports
  |-- report generation
  |-- low-stock alerts
```

Decision: begin as a modular monolith. Extract a worker when asynchronous workloads justify it. Split identity into a separately deployed service only when a real ownership/deployment/security need outweighs the extra network and operations burden.

7. Day 6 — Capstone and assessment

Work without following a tutorial line-by-line. You may use documentation, but record what you had to look up.

Implementation assessment

Task	Pass condition	Score
API contract	Valid schemas, pagination, consistent errors, documented endpoints	10
Data model	Constraints, tenant-scoped uniqueness, sensible indexes, migrations	15
Stock correctness	Ledger and balance update atomically; quantity cannot go negative	20
Concurrency	Parallel stock issues do not oversell; test is repeatable	15
Idempotency	Duplicate key does not apply a stock change twice	10
Testing	Unit + PostgreSQL integration tests cover key failures	10
System design	Clear diagram, data flow, failure modes, trade-offs	10
Engineering quality	README, CI, logs/health, clean module boundaries	10
Total		100

Readiness interpretation

- 85–100: proceed to the next project phase; revisit any correctness issue before adding features.
- 70–84: proceed cautiously but spend a focused repair session on the lowest-scoring area.
- Below 70: repeat the weak days; do not add microservices to compensate for fundamentals.
- Automatic fail regardless of score: cross-tenant data exposure, negative stock, duplicate movement on retry, or tests that only pass with mocked database behavior.

Day 7 — Recovery, consolidation, and catch-up

- Take a proper break; do not schedule another 12-hour sprint.
- Re-run the whole test suite from a clean checkout or clean Docker volume.
- Review failed tests and your gap list; write a one-page “what I learned / what I still cannot explain.”
- Prepare the architecture diagram and a 5-minute verbal explanation of the transaction and tenant isolation.
- Tag the week-one checkpoint in Git. Start the larger ERP build only after the critical invariants pass.

8. Concepts to know cold

Concept	Interview-ready explanation prompt
Authentication vs authorization	Who are you? vs what are you allowed to do?
Constraint vs application check	Which rule must remain true even if a bug or race bypasses app logic?
Index	Which query does it speed up, what is the write/storage cost, and how will you verify?
Transaction	Which writes must succeed or fail together?
Isolation / lock	What concurrent interleaving could break correctness?
Idempotency	What if a client times out after the server commits and retries?
Unit vs integration test	What boundary is exercised, and what failure would a mock hide?
Modular monolith	How do you enforce module boundaries without network calls?
Microservice	Who owns its data, how are failures handled, and how does it deploy independently?
Queue	How are duplicate jobs, retries, poison messages, and backlog handled?
Tenant isolation	Can a valid user from tenant A access tenant B by changing an ID?
Observability	How will you distinguish a slow DB from a slow worker or external dependency?

Security and reliability checklist

- Never trust tenant_id, role, price, or quantity supplied by the client without server-side authorization and validation.
- Use parameterized queries/ORM parameters; never concatenate untrusted values into SQL.
- Do not log passwords, access tokens, secrets, or sensitive customer data.
- Use least-privilege database credentials and separate local/test/production configuration.
- Set request body size, pagination, and timeout limits.
- Return generic external errors; log actionable internal details with a request ID.
- Use constraints and transactions for business invariants, not just controller checks.
- Plan for backups and restore tests before calling a production system reliable.

Architecture decision record (ADR) template

```
# ADR-001: Start as a modular monolith

Status: Accepted
Context:
Decision:
Alternatives considered:
Consequences / trade-offs:
Security and failure implications:
Revisit when:
```

Recommended official references

Use these to verify behavior and learn from primary documentation; do not attempt to read all documentation end-to-end in one week.

- Node.js documentation: https://nodejs.org/docs/latest/api/
- TypeScript handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- PostgreSQL documentation: https://www.postgresql.org/docs/current/
- Prisma documentation: https://www.prisma.io/docs
- Vitest: https://vitest.dev/guide/
- Docker Compose: https://docs.docker.com/compose/

- OpenAPI specification: https://spec.openapis.org/oas/latest.html

---

End-of-week deliverables: a working inventory API, PostgreSQL migrations, concurrency and idempotency tests, CI, an ERD, an architecture diagram, an ADR, and a short honest gap assessment.